

Homework 4: Spatial and Temporal Blocking for Model Evaluation

GEOL 599: Machine Learning

Contents

Goal	2
1. Load the Sea Surface Temperature Dataset	2
Discussion	3
2. Explore the Data Visually	3
TODO: Describe the Correlation Structure	5
3. Convert Gridded Data to a Machine-Learning Table	5
Quick Table Checks	6
TODO: Additional Exploration	7
TODO: Historical Monthly SST Feature	8
4. Define the Prediction Problem	9
5. Create a Reusable Model Workflow	9
6. Naive Evaluation: Random k-Fold Cross-Validation	11
Discussion: Random CV	11
7. Temporal Block Cross-Validation	11
TODO: Summarize Temporal CV	12
Discussion: Temporal Blocking	12
8. Spatial Block Cross-Validation	13
TODO: Variogram of Residuals	13
Create Spatial Blocks	14
Visualize the Spatial Blocks	15
Coding Task: Spatial Block CV	16
Coding Task: Summarize Spatial CV	17
Discussion: Spatial Blocking	17
9. Spatial-Temporal Holdout	17
Discussion: Spatial-Temporal Holdout	19
10. Compare All Evaluation Strategies	19
Plot the Comparison	19
11. Final Reflection	20

```
# If needed, install these packages first:
# install.packages(c("tidyverse", "tidymodels", "terra", "ranger", "gstat", "sf", "sp"))

library(tidyverse)
```

```
library(tidymodels)
library(terra)
library(ranger)
library(gstat)
library(sf)
library(sp)

library(ggplot2)

tidymodels_prefer()
set.seed(42)
```

Goal

Compare ordinary random cross-validation with blocked cross-validation for spatially and temporally correlated environmental data.

In this homework, you will use a gridded **sea surface temperature (SST)** dataset. SST is a good example because nearby ocean grid cells tend to have similar temperatures, and nearby months or years are also related. This means ordinary random cross-validation can give an overly optimistic estimate of model performance.

Sections marked **TODO** are places where you should write or modify code.

1. Load the Sea Surface Temperature Dataset

The data are stored as a NetCDF file, a common format for gridded environmental and oceanographic data. Instead of rows and columns only, the data have dimensions such as `time`, `lat`, and `lon`.

```
data_url <- "https://raw.githubusercontent.com/dblaskey/ML_Course_Code/main/Data/sst.nc"
file_path <- "sst.nc"

if (!file.exists(file_path)) {
  download.file(data_url, destfile = file_path, mode = "wb")
}

sst_rast <- rast(file_path)
sst_rast
```

```
## class      : SpatRaster
## size       : 31, 43, 116 (nrow, ncol, nlyr)
## dimensions  : lon, lat, time (43, 31, 116)
## resolution  : 4, 4 (x, y)
## extent     : 118, 290, -62, 62 (xmin, xmax, ymin, ymax)
## coord. ref. : lon/lat WGS 84 (CRS84) (OGC:CRS84)
## source      : sst.nc
## names       : sst_1, sst_2, sst_3, sst_4, sst_5, sst_6, ...
## unit        : degC
## time (days) : 1992-01-01 to 2020-10-01 (116 steps)
```

The SST variable is stored as a multilayer raster. Each layer represents one time step.

```
nlyr(sst_rast)
```

```
## [1] 116
```

```

ext(sst_rast)

## SpatExtent : 118, 290, -62, 62 (xmin, xmax, ymin, ymax)

res(sst_rast)

## [1] 4 4

time(sst_rast)

## [1] "1992-01-01" "1992-04-01" "1992-07-01" "1992-10-01" "1993-01-01"
## [6] "1993-04-01" "1993-07-01" "1993-10-01" "1994-01-01" "1994-04-01"
## [11] "1994-07-01" "1994-10-01" "1995-01-01" "1995-04-01" "1995-07-01"
## [16] "1995-10-01" "1996-01-01" "1996-04-01" "1996-07-01" "1996-10-01"
## [21] "1997-01-01" "1997-04-01" "1997-07-01" "1997-10-01" "1998-01-01"
## [26] "1998-04-01" "1998-07-01" "1998-10-01" "1999-01-01" "1999-04-01"
## [31] "1999-07-01" "1999-10-01" "2000-01-01" "2000-04-01" "2000-07-01"
## [36] "2000-10-01" "2001-01-01" "2001-04-01" "2001-07-01" "2001-10-01"
## [41] "2002-01-01" "2002-04-01" "2002-07-01" "2002-10-01" "2003-01-01"
## [46] "2003-04-01" "2003-07-01" "2003-10-01" "2004-01-01" "2004-04-01"
## [51] "2004-07-01" "2004-10-01" "2005-01-01" "2005-04-01" "2005-07-01"
## [56] "2005-10-01" "2006-01-01" "2006-04-01" "2006-07-01" "2006-10-01"
## [61] "2007-01-01" "2007-04-01" "2007-07-01" "2007-10-01" "2008-01-01"
## [66] "2008-04-01" "2008-07-01" "2008-10-01" "2009-01-01" "2009-04-01"
## [71] "2009-07-01" "2009-10-01" "2010-01-01" "2010-04-01" "2010-07-01"
## [76] "2010-10-01" "2011-01-01" "2011-04-01" "2011-07-01" "2011-10-01"
## [81] "2012-01-01" "2012-04-01" "2012-07-01" "2012-10-01" "2013-01-01"
## [86] "2013-04-01" "2013-07-01" "2013-10-01" "2014-01-01" "2014-04-01"
## [91] "2014-07-01" "2014-10-01" "2015-01-01" "2015-04-01" "2015-07-01"
## [96] "2015-10-01" "2016-01-01" "2016-04-01" "2016-07-01" "2016-10-01"
## [101] "2017-01-01" "2017-04-01" "2017-07-01" "2017-10-01" "2018-01-01"
## [106] "2018-04-01" "2018-07-01" "2018-10-01" "2019-01-01" "2019-04-01"
## [111] "2019-07-01" "2019-10-01" "2020-01-01" "2020-04-01" "2020-07-01"
## [116] "2020-10-01"

```

Discussion

Look at the dataset metadata above.

Questions to answer:

1. What are the dimensions of the dataset? The dimensions of this dataset are lon, lat, time in 43, 31, 116
2. What is the spacing between latitude and longitude grid cells? The spatial resolution is a 4x4 deg grid, so 4 degrees between each latitude and longitude grid cell
3. How many time steps are included? The temporal resolution is 1992 through 2020, at three month intervals. Each time point is the 1st of the month for January, April, July, and October.
4. Why might neighboring grid cells have similar SST values? Spatial auto-correlation and large-scale physical drivers could cause lack of variation in surface temperature (little surface wind, no upwelling).

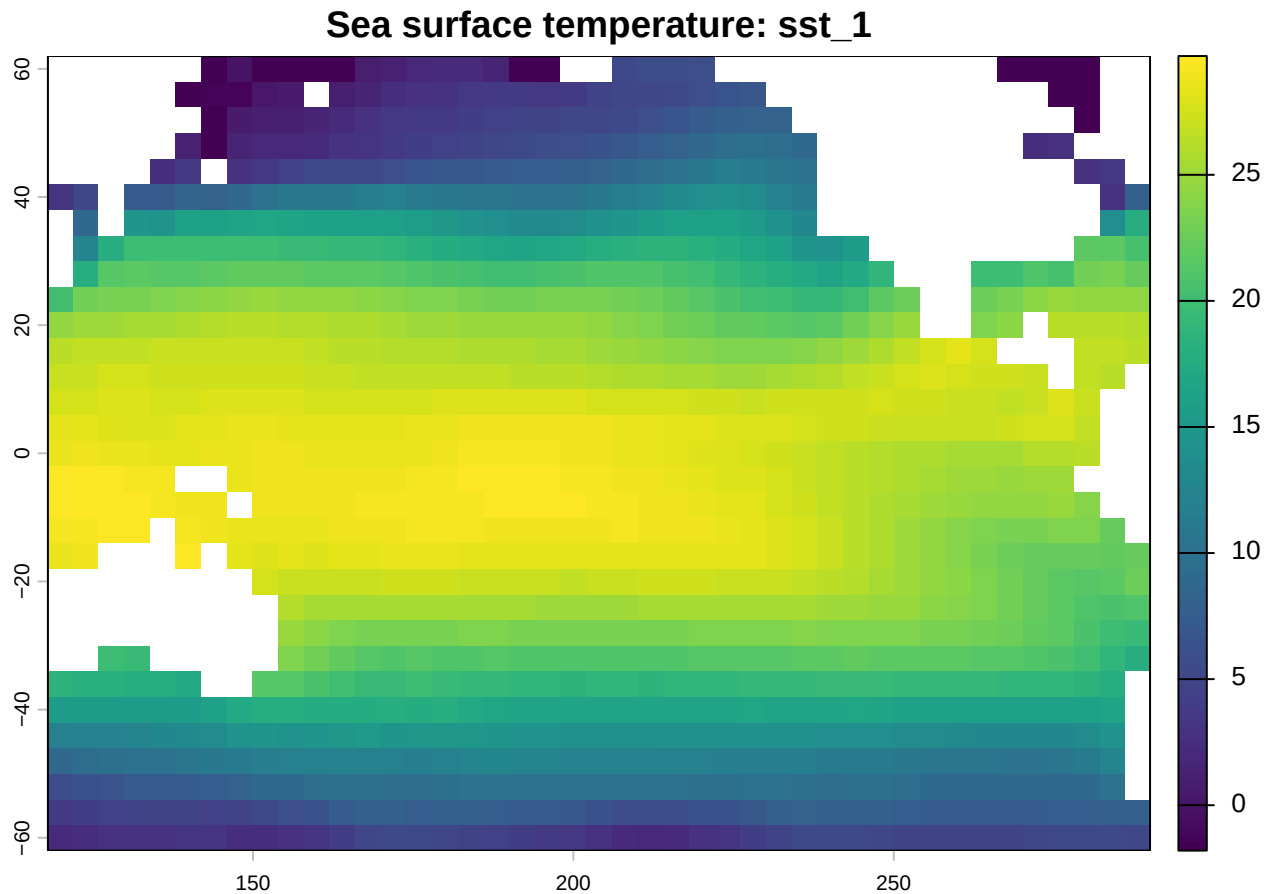
Your answer: TODO

2. Explore the Data Visually

Before modeling, inspect the target variable. SST has strong geographic structure and seasonal/temporal structure. The first plot shows the SST field for one time step.

```
one_time <- sst_rast[[1]]
```

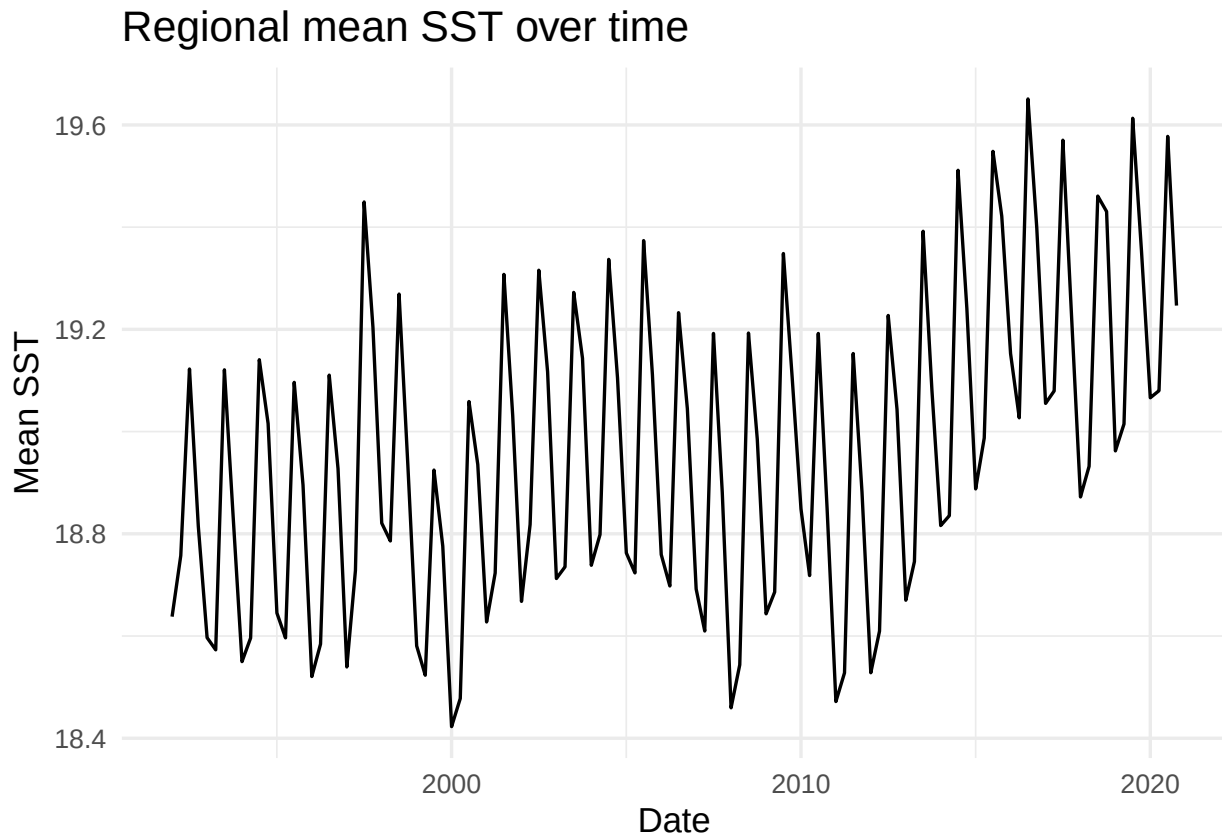
```
plot(one_time, main = paste("Sea surface temperature:", names(one_time)))
```



The next plot averages SST across the selected region for each time step.

```
regional_mean <- global(sst_rast, fun = "mean", na.rm = TRUE) %>%
  as_tibble() %>%
  mutate(
    layer = row_number(),
    date = time(sst_rast)
  )

ggplot(regional_mean, aes(x = date, y = mean)) +
  geom_line() +
  labs(
    title = "Regional mean SST over time",
    x = "Date",
    y = "Mean SST"
  ) +
  theme_minimal(base_size = 13)
```



TODO: Describe the Correlation Structure

Write a short answer after running the plots.

Prompt: Based on the maps and time series, where do you expect random cross-validation to be misleading? Consider both nearby locations and nearby times.

Your answer: TODO Random cross-validation can create unbalanced folds for seasons and spatial extents. I expect random cross-validation would be misleading around the equator, as temperatures will be more consistent regardless of season and temperate regions could align with lower/higher latitudes when they are in their hottest/coldest season. I am specifically thinking the western/warm and eastern/cool boundary currents being randomly selected and misleading the model towards the equator and poles. We also have spatial autocorrelation to account for and the temporal oscillation between the hemispheres.

3. Convert Gridded Data to a Machine-Learning Table

Most `tidymodels` workflows expect a table where each row is one observation. Here, one row will represent one grid cell at one time. The target is `sst`. The predictors will be simple features based on location, season, and time.

```
# Convert raster layers to a wide table with x/y columns and one column per time step.
sst_wide <- as.data.frame(sst_rast, xy = TRUE, na.rm = FALSE) %>%
  as_tibble() %>%
  rename(lon = x, lat = y)

# Use raster time metadata when available.
sst_dates <- time(sst_rast)
```

```
# Pivot from wide raster format to long machine-learning format.
df <- sst_wide %>%
  pivot_longer(
    cols = -c(lon, lat),
    names_to = "layer",
    values_to = "sst"
  ) %>%
  drop_na(sst) %>%
  mutate(
    layer_index = as.integer(factor(layer, levels = names(sst_rast))),
    time = sst_dates[layer_index],
    year = lubridate::year(time),
    month = lubridate::month(time),
    time_index = (year - min(year, na.rm = TRUE)) * 12 + (month - 1),
    month_sin = sin(2 * pi * month / 12),
    month_cos = cos(2 * pi * month / 12),
    lon_sin = sin(2 * pi * lon / 360),
    lon_cos = cos(2 * pi * lon / 360)
  )

dim(df)
```

```
## [1] 131544      13
```

```
head(df)
```

```
## # A tibble: 6 x 13
##   lon  lat layer  sst layer_index time      year month time_index
##   <dbl> <dbl> <chr> <dbl>      <int> <date>    <dbl> <dbl>      <dbl>
## 1  144   60 sst_1 -1.80          1 1992-01-01 1992     1         0
## 2  144   60 sst_2 -1.78          2 1992-04-01 1992     4         3
## 3  144   60 sst_3 10.0          3 1992-07-01 1992     7         6
## 4  144   60 sst_4  5.82          4 1992-10-01 1992    10         9
## 5  144   60 sst_5 -1.80          5 1993-01-01 1993     1        12
## 6  144   60 sst_6 -1.80          6 1993-04-01 1993     4        15
## # i 4 more variables: month_sin <dbl>, month_cos <dbl>, lon_sin <dbl>,
## #   lon_cos <dbl>
```

Quick Table Checks

Use this section to understand the rows that the machine-learning model will see.

```
cat("Number of rows:", nrow(df), "\n")
```

```
## Number of rows: 131544
```

```
cat("Number of years:", n_distinct(df$year), "\n")
```

```
## Number of years: 29
```

```
cat("Years:", min(df$year), "to", max(df$year), "\n")
```

```
## Years: 1992 to 2020
```

```
cat("Number of grid locations:", n_distinct(paste(df$lat, df$lon)), "\n")
```

```
## Number of grid locations: 1134
```

```
summary(df$sst)
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##    -1.80   11.02   21.48   18.96   27.17   31.33
```

TODO: Additional Exploration

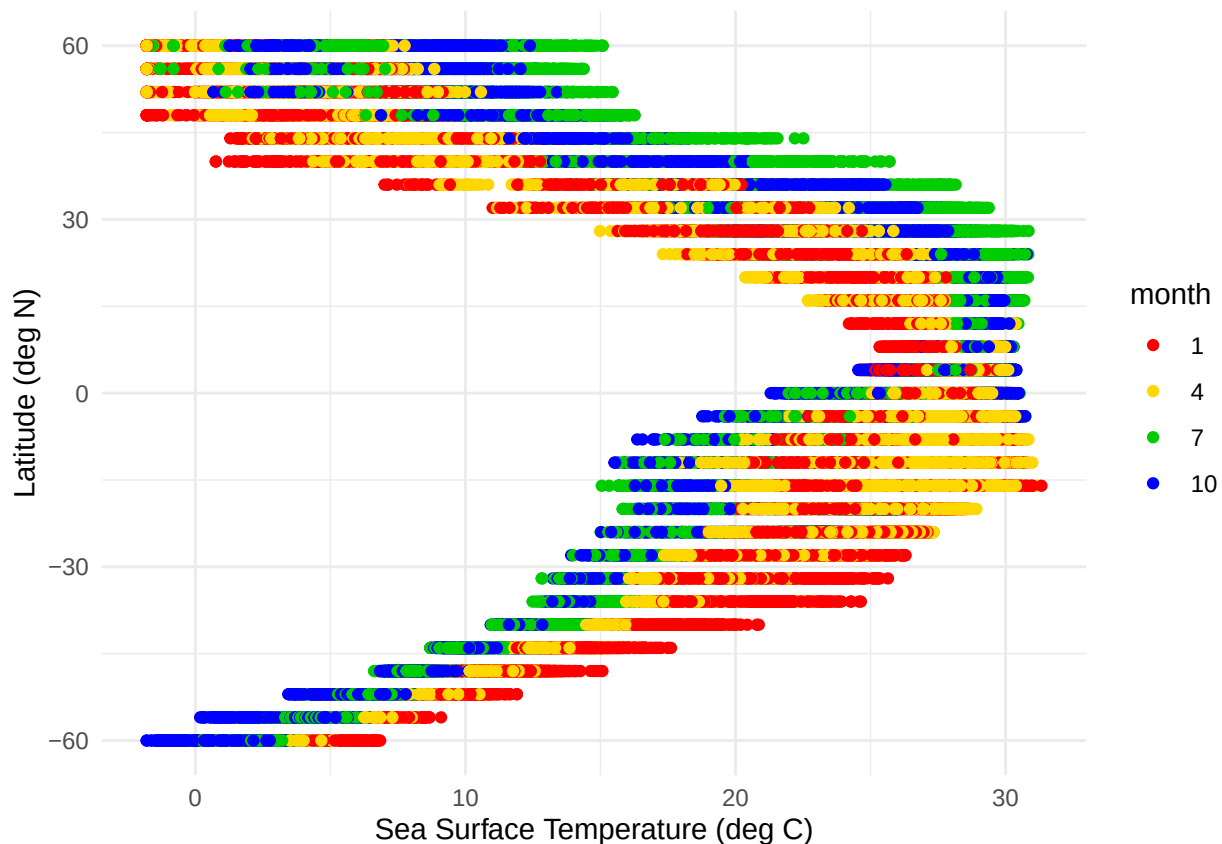
Add at least two exploratory checks of your own.

Examples:

- Plot SST versus latitude.
- Plot SST versus month.
- Compare SST distributions for different years.
- Map a different time step.
- Compute average SST by latitude band.

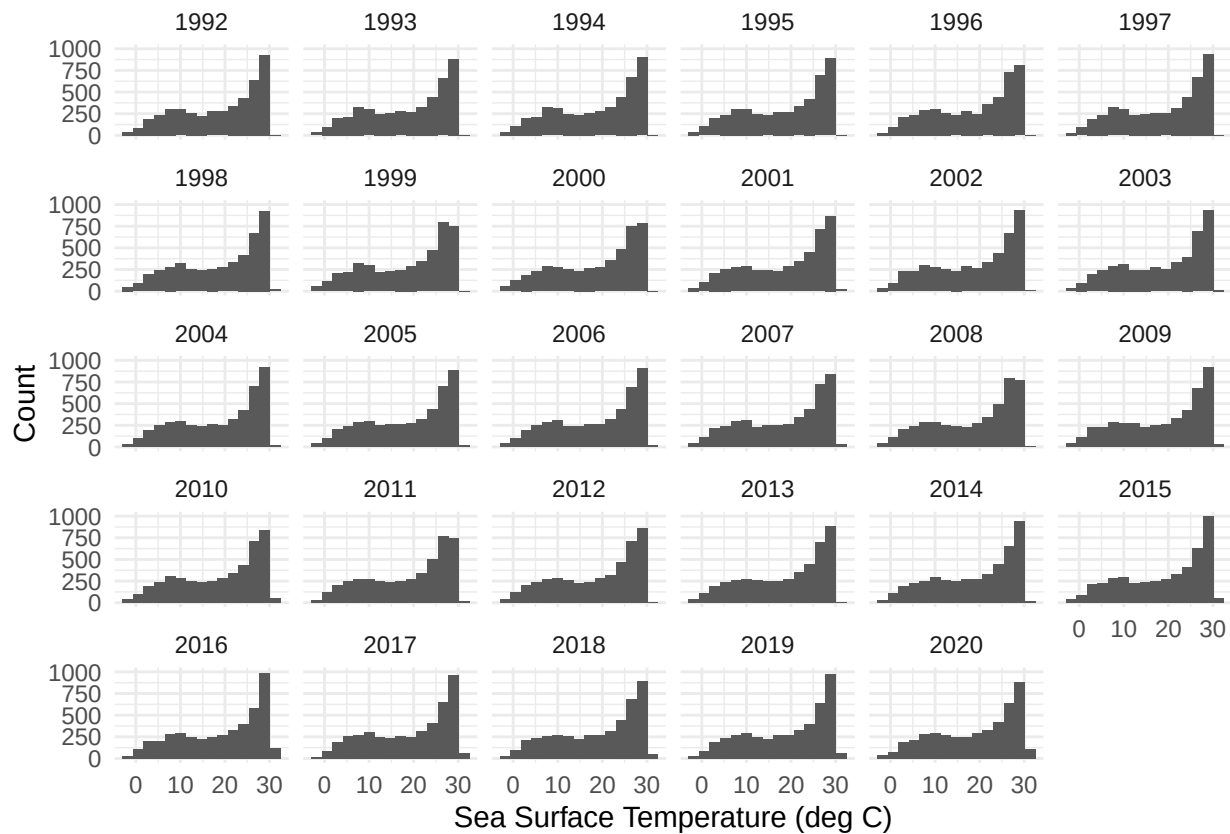
```
# TODO: Add at least two exploratory checks of your own.
```

```
df2=df
df2$month=as.factor(df2$month)
ggplot()+
  geom_point(data=df2,aes(x=sst,y=lat,color=month))+
  labs(x="Sea Surface Temperature (deg C)", y="Latitude (deg N)")+
  scale_color_manual(values=c("1"="red","4"="gold","7"="green3","10"="blue"))+
  theme_minimal()
```



```
ggplot()+
  geom_histogram(data=df,aes(x=sst),bins=15)+
```

```
facet_wrap(~year)+
labs(x="Sea Surface Temperature (deg C)", y="Count")+
theme_minimal()
```



TODO: Historical Monthly SST Feature

Create a historical average SST feature for each location and month. Be careful not to use future data when creating this column.

The idea is:

1. Group by location and month.
2. Sort by time.
3. Compute the cumulative mean from previous observations only.
4. Use `lag()` so the current observation is not used to predict itself.

```
# TODO: Create Month_Average_sst without using future data.
# Hint: use lag(cummean(sst)).
```

```
# Example scaffold:
```

```
df <- df %>%
  group_by(lat, lon, month) %>%
  mutate(Month_Average_sst = dplyr::lag(cummean(sst))) %>%
  ungroup()
```

```
df <- df %>%
  drop_na(Month_Average_sst)
```

4. Define the Prediction Problem

We will predict SST from simple location and time features. This is not meant to be the best possible ocean forecasting model. The goal is to study how evaluation changes when data are spatially and temporally correlated.

```
feature_cols <- c(
  "lat",
  "lon_sin",
  "lon_cos",
  "month_sin",
  "month_cos",
  "time_index",
  "Month_Average_sst"
)

target_col <- "sst"

# If Month_Average_sst does not exist yet, stop with a helpful reminder.
stopifnot("Month_Average_sst" %in% names(df))

model_data <- df %>%
  select(all_of(c(target_col, feature_cols, "year", "lat", "lon", "time"))) %>%
  mutate(row_id = row_number())

head(model_data)
```

```
## # A tibble: 6 x 12
##   sst   lat lon_sin lon_cos month_sin month_cos time_index Month_Average_sst
##   <dbl> <dbl>   <dbl>   <dbl>   <dbl>   <dbl>   <dbl>         <dbl>
## 1 -1.80    60  0.588  -0.809    0.5    0.866    12         -1.80
## 2 -1.80    60  0.588  -0.809    0.866   -0.5    15         -1.78
## 3 11.9     60  0.588  -0.809   -0.50   -0.866    18          10.0
## 4  6.23    60  0.588  -0.809   -0.866    0.5    21           5.82
## 5 -1.80    60  0.588  -0.809    0.5    0.866    24         -1.80
## 6 -1.80    60  0.588  -0.809    0.866   -0.5    27         -1.79
## # i 4 more variables: year <dbl>, lon <dbl>, time <date>, row_id <int>
```

5. Create a Reusable Model Workflow

We will use a tree-based regression model. Tree-based models can capture nonlinear patterns such as warmer temperatures near the equator and seasonal variation.

In tidymodels, we define:

- a **recipe** for preprocessing,
- a **model specification**, and
- a **workflow** that combines them.

```
sst_recipe <- recipe(sst ~ lat + lon_sin + lon_cos + month_sin + month_cos + time_index + Month_Average_sst,
  data = model_data) %>%
  step_impute_median(all_numeric_predictors())

rf_spec <- rand_forest(
  trees = 300,
  min_n = 5,
```

```

  mtry = 4
) %>%
  set_mode("regression") %>%
  set_engine("ranger", importance = "impurity")

sst_workflow <- workflow() %>%
  add_recipe(sst_recipe) %>%
  add_model(rf_spec)

sst_workflow

## == Workflow =====
## Preprocessor: Recipe
## Model: rand_forest()
##
## -- Preprocessor -----
## 1 Recipe Step
##
## * step_impute_median()
##
## -- Model -----
## Random Forest Model Specification (regression)
##
## Main Arguments:
##   mtry = 4
##   trees = 300
##   min_n = 5
##
## Engine-Specific Arguments:
##   importance = impurity
##
## Computational engine: ranger

summarize_resamples <- function(resample_results, label) {
  collect_metrics(resample_results) %>%
    select(.metric, mean, std_err) %>%
    pivot_wider(
      names_from = .metric,
      values_from = c(mean, std_err)
    ) %>%
    transmute(
      Evaluation = label,
      `Mean RMSE` = mean_rmse,
      `SE RMSE` = std_err_rmse,
      `Mean MAE` = mean_mae,
      `Mean R2` = mean_rsqr
    )
}

regression_metrics <- metric_set(rmse, mae, rsqr)

```

6. Naive Evaluation: Random k-Fold Cross-Validation

Random k-fold cross-validation mixes observations from different locations and times into each fold. This is common in introductory machine learning, but it is often too optimistic for spatial-temporal data because very similar observations may appear in both training and test folds.

```
random_cv <- vfold_cv(model_data, v = 5)

random_results <- fit_resamples(
  sst_workflow,
  resamples = random_cv,
  metrics = regression_metrics,
  control = control_resamples(save_pred = TRUE)
)

collect_metrics(random_results)

## # A tibble: 3 x 6
##   .metric .estimator mean      n std_err .config
##   <chr>   <chr>     <dbl> <int>   <dbl> <chr>
## 1 mae     standard  0.332     5 0.00123 pre0_mod0_post0
## 2 rmse    standard  0.463     5 0.00240 pre0_mod0_post0
## 3 rsq     standard  0.997     5 0.0000292 pre0_mod0_post0

random_summary <- summarize_resamples(random_results, "Random k-fold CV")
random_summary

## # A tibble: 1 x 5
##   Evaluation      `Mean RMSE` `SE RMSE` `Mean MAE` `Mean R2`
##   <chr>          <dbl>    <dbl>    <dbl>    <dbl>
## 1 Random k-fold CV      0.463    0.00240    0.332    0.997
```

Discussion: Random CV

Questions:

1. What is the average RMSE under random k-fold CV? Average RMSE is 0.4633453
2. Why might this number be too optimistic? There are spatial and temporal auto correlations that have not been accounted for and based on previous questions, I assume this leans towards a 'better' performing model than the data truly is showing (because thats how all physical oceanography data acts).
3. In this dataset, what kinds of information can leak from the training folds into the test fold? Data leaks can be from surrounding points and depending on the preprocessing that happened to the data (before we got it) there could be interpolation for missing values. I think also randomly selecting data from later months in the year and can leak data if earlier points are excluded.

Your answer: TODO

7. Temporal Block Cross-Validation

Now we will hold out entire years. This asks a different question:

Can the model generalize to years it has not seen during training?

This is a better approximation of forecasting or prediction into future time periods than random cross-validation.

```
# TODO: Complete temporal block cross-validation.

# Step 1: create grouped folds by year.
temporal_cv <- group_vfold_cv(model_data,group=year,v=5) # replace NULL with group_vfold_cv(...)

# Step 2: run blocked cross-validation.
temporal_results <- fit_resamples(
  sst_workflow,
  resamples = temporal_cv,
  metrics = regression_metrics,
  control = control_resamples(save_pred = TRUE)
) # replace NULL with fit_resamples(...)

# Step 3: display the results.
collect_metrics(temporal_results)
```

```
## # A tibble: 3 x 6
##   .metric .estimator mean      n std_err .config
##   <chr>   <chr>      <dbl> <int>   <dbl> <chr>
## 1 mae     standard    0.438     5 0.0168 pre0_mod0_post0
## 2 rmse    standard    0.596     5 0.0232 pre0_mod0_post0
## 3 rsq     standard    0.996     5 0.000344 pre0_mod0_post0
```

TODO: Summarize Temporal CV

Use `summarize_resamples()` to create a one-row summary for temporal block CV.

```
# TODO: Summarize your temporal CV results.
# random_summary <- summarize_resamples(random_results, "Random k-fold CV")
temporal_summary <- summarize_resamples(temporal_results, "Blocked Temporal k-fold CV")

temporal_summary
```

```
## # A tibble: 1 x 5
##   Evaluation                `Mean RMSE` `SE RMSE` `Mean MAE` `Mean R2`
##   <chr>                  <dbl>      <dbl>    <dbl>    <dbl>
## 1 Blocked Temporal k-fold CV    0.596    0.0232    0.438    0.996
```

Discussion: Temporal Blocking

Questions:

1. Is temporal block CV better or worse than random CV? Temporal block performs better than the random CV!
2. What does the difference tell you? Random CV's does not account for temporal autocorrelation, but when accounted the model fits better.
3. Does the model appear to generalize well to unseen years? There is a decent ability to predict unseen years but does not perform as well as random k-fold model based on R2.
4. Would this be a fair test if your real goal were to predict future years? I think this would be a fair test if were were to predict future seasonally averaged Pacific surface temperature

Your answer: TODO

8. Spatial Block Cross-Validation

Now we will hold out entire regions. This asks:

Can the model generalize to locations it has not seen during training?

Spatial blocking is useful when your real prediction task involves mapping unsampled locations or transferring a model to a new region.

TODO: Variogram of Residuals

Use cross-validated residuals from the random CV model to estimate the spatial scale of residual autocorrelation. This avoids using purely in-sample residuals, which can understate model error.

Hint: Look back at the earlier spatial regression activity. One reasonable approach is:

1. Fit the workflow once to the full modeling table.
2. Predict SST for the same rows.
3. Compute residuals.
4. Convert one time step, or a sample of residuals, into an `sf/sp` object.
5. Use `gstat::variogram()`.

TODO: Create a variogram of model residuals to estimate spatial autocorrelation distance.

```
random_predictions <- collect_predictions(random_results)

residual_data <- random_predictions %>%
  mutate(residual = sst - .pred) %>%
  left_join(
    model_data %>%
      mutate(.row = row_number()) %>%
      select(.row, lat, lon, time),
    by = ".row"
  )

residual_one_time <- residual_data %>%
  filter(time == min(time)) %>%
  sample_n(size = min(3000, n()))

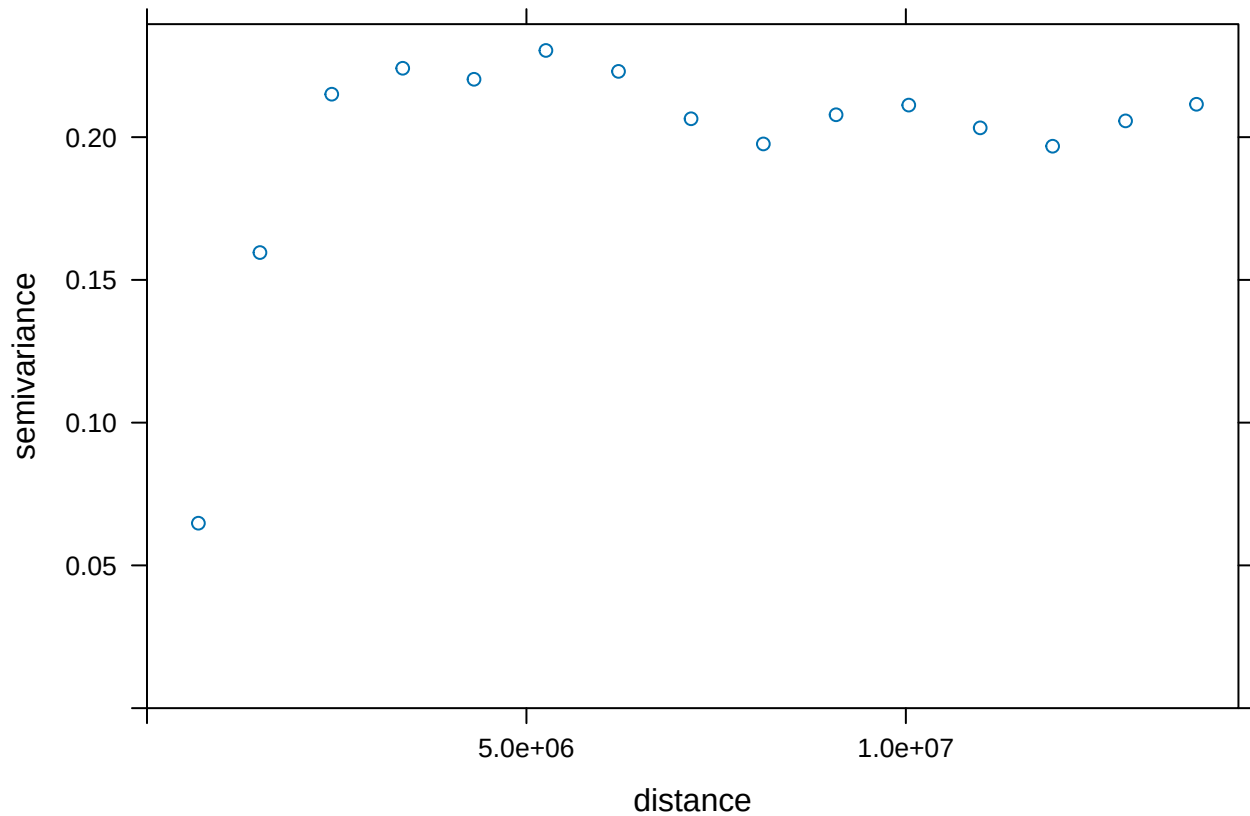
residual_sf <- st_as_sf(
  residual_one_time,
  coords = c("lon", "lat"),
  crs = 4326
) %>%
  st_transform(3857)

residual_sp <- as(residual_sf, "Spatial")

residual_variogram <- gstat::variogram(
  residual ~ 1,
  data = residual_sp
)
#TODO: Use variogram()

plot(residual_variogram, main = "Variogram of cross-validated residuals")
```

Variogram of cross-validated residuals



```
residual_variogram$dist[which.max(residual_variogram$gamma)]
```

```
## [1] 5256688
```

Create Spatial Blocks

Once you have estimated the extent of spatial autocorrelation, use that information to set the block size.

```
# TODO: choose block sizes based on your variogram or exploratory reasoning.  
# chosen based on distance with max semivariance (5.2X10^6)
```

```
lat_block_size <- 5.2 # replace NULL with a numeric block size, such as 5  
lon_block_size <- 5.2 # replace NULL with a numeric block size, such as 5
```

```
model_data <- model_data %>%  
  mutate(  
    lat_block = floor((lat + 90) / lat_block_size),  
    lon_block = floor(lon / lon_block_size),  
    spatial_block = paste(lat_block, lon_block, sep = "_")  
  )  
  
cat("Number of spatial blocks:", n_distinct(model_data$spatial_block), "\n")
```

```
## Number of spatial blocks: 691
```

```
model_data %>%  
  select(lat, lon, lat_block, lon_block, spatial_block) %>%
```

```
distinct() %>%
head()
```

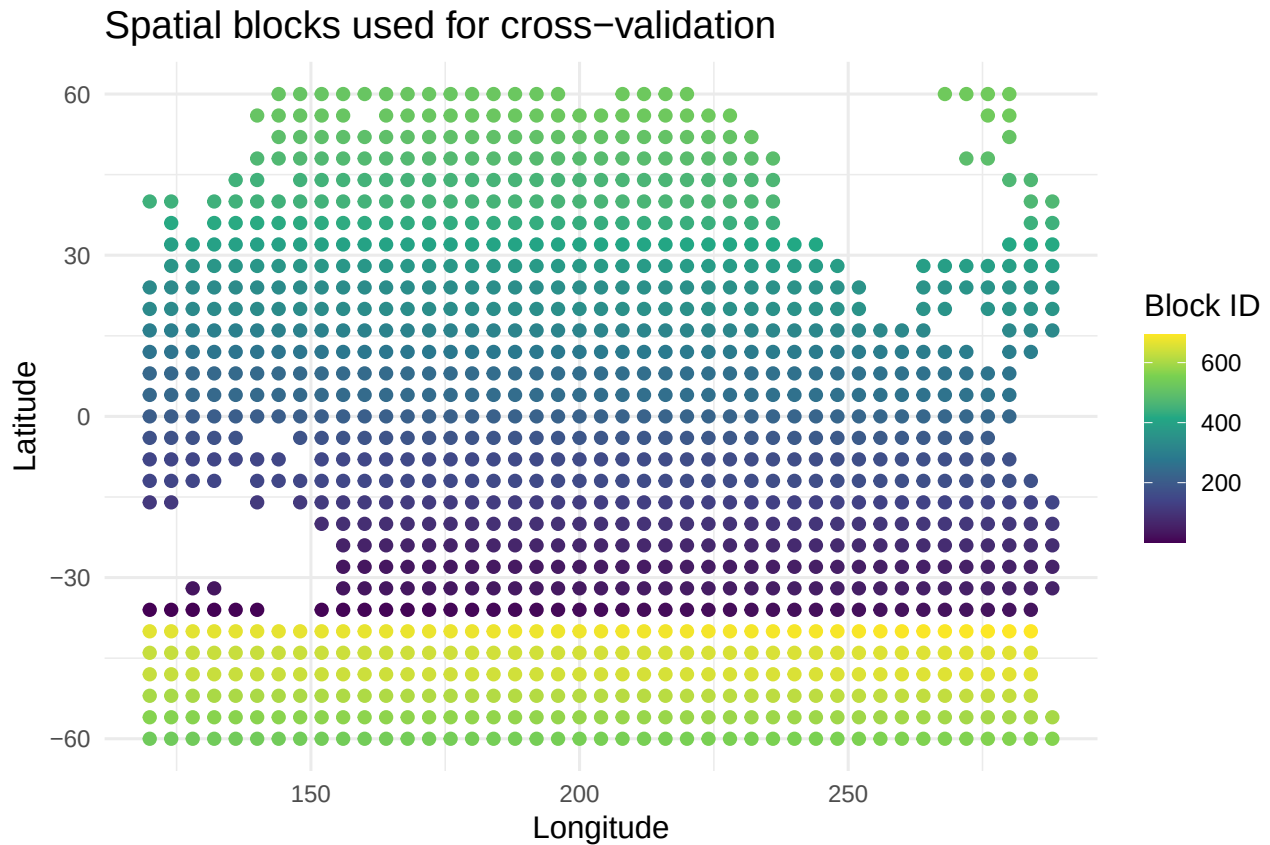
```
## # A tibble: 6 x 5
##   lat lon lat_block lon_block spatial_block
##   <dbl> <dbl>   <dbl>   <dbl> <chr>
## 1    60   144     28     27 28_27
## 2    60   148     28     28 28_28
## 3    60   152     28     29 28_29
## 4    60   156     28     30 28_30
## 5    60   160     28     30 28_30
## 6    60   164     28     31 28_31
```

Visualize the Spatial Blocks

Each point below is one grid location. The colors represent block IDs, not physical values.

```
block_map <- model_data %>%
  distinct(lat, lon, spatial_block) %>%
  mutate(block_code = as.integer(factor(spatial_block)))

ggplot(block_map, aes(x = lon, y = lat, color = block_code)) +
  geom_point(size = 2) +
  coord_equal() +
  scale_color_viridis_c() +
  labs(
    title = "Spatial blocks used for cross-validation",
    x = "Longitude",
    y = "Latitude",
    color = "Block ID"
  ) +
  theme_minimal(base_size = 13)
```



Coding Task: Spatial Block CV

Complete the spatial block cross-validation.

Hints:

- Use `group_vfold_cv()` again.
- The grouping variable should be `spatial_block`.
- The block columns are used only to define folds; they are not predictors in the recipe.

```
# TODO: Complete spatial block cross-validation.

# Step 1: create grouped folds by spatial block.
spatial_cv <- group_vfold_cv(model_data, group=spatial_block, v=5) # replace NULL with group_vfold_cv(...)

# Step 2: run blocked cross-validation.
spatial_results <- fit_resamples(
  sst_workflow,
  resamples = spatial_cv,
  metrics = regression_metrics,
  control = control_resamples(save_pred = TRUE)
) # replace NULL with fit_resamples(...)

# Step 3: display the results.
collect_metrics(spatial_results)
```

```
## # A tibble: 3 x 6
##   .metric .estimator mean     n std_err .config
```

```
##   <chr>   <chr>      <dbl> <int>    <dbl> <chr>
## 1 mae     standard    0.338    5 0.00573 pre0_mod0_post0
## 2 rmse     standard    0.472    5 0.0100  pre0_mod0_post0
## 3 rsq      standard    0.997    5 0.000129 pre0_mod0_post0
```

Coding Task: Summarize Spatial CV

Use `summarize_resamples()` to create a one-row summary for spatial block CV.

```
# TODO: Summarize your spatial CV results.
```

```
spatial_summary <- summarize_resamples(spatial_results, "Blocked Spatial k-fold CV")
```

```
spatial_summary
```

```
## # A tibble: 1 x 5
##   Evaluation                `Mean RMSE` `SE RMSE` `Mean MAE` `Mean R2`
##   <chr>                  <dbl>      <dbl>      <dbl>      <dbl>
## 1 Blocked Spatial k-fold CV    0.472    0.0100    0.338    0.997
```

Discussion: Spatial Blocking

Questions:

1. Is spatial block CV better or worse than random CV? The spatial block was better but not by much than the random CV (by mean RMSE).
2. What does this say about the model's ability to predict in new locations? This model can predict new locations as well as the random k-fold model, which is very good at 0.997 R2 value.
3. Why is latitude likely such a powerful predictor of SST? Higher latitudes (closer to the poles) receive less sunlight and are more likely to be cold/experience variable temperatures than the equator that gets consistent amounts of solar radiation. The degree of latitude determines the variability of temperature in these regions and can be averaged globally.
4. Why might a model still perform well in spatial blocks if the region has a strong latitudinal gradient? Latitude is the main predictor for this model, so a strong latitudinal gradient means a wider area to predict. Longitude has very little relationship to sea surface temperature (especially satellite products which perform poorly close to the coast)

Your answer: TODO

9. Spatial-Temporal Holdout

Spatial blocking asks whether the model can generalize to new places. Temporal blocking asks whether the model can generalize to new years. A spatial-temporal holdout combines both ideas.

Here we create a stricter test: the test data must come from held-out spatial blocks **and** held-out years. The training data exclude those spatial blocks and those years.

```
make_strict_spatiotemporal_splits <- function(data, spatial_col = "spatial_block", time_col = "year", v)
  spatial_blocks <- sort(unique(data[[spatial_col]]))
  time_blocks <- sort(unique(data[[time_col]]))

  spatial_fold_ids <- sample(rep(seq_len(v), length.out = length(spatial_blocks)))
  time_fold_ids <- sample(rep(seq_len(v), length.out = length(time_blocks)))

  spatial_lookup <- tibble(
```

```

    spatial_block_value = spatial_blocks,
    spatial_fold = spatial_fold_ids
  )

time_lookup <- tibble(
  time_block_value = time_blocks,
  time_fold = time_fold_ids
)

split_list <- vector("list", v)

for (fold_id in seq_len(v)) {
  heldout_spatial <- spatial_lookup %>%
    filter(spatial_fold == fold_id) %>%
    pull(spatial_block_value)

  heldout_time <- time_lookup %>%
    filter(time_fold == fold_id) %>%
    pull(time_block_value)

  assessment_idx <- which(data[[spatial_col]] %in% heldout_spatial & data[[time_col]] %in% heldout_time)
  analysis_idx <- which(!(data[[spatial_col]] %in% heldout_spatial) & !(data[[time_col]] %in% heldout_time))

  split_list[[fold_id]] <- make_splits(
    list(analysis = analysis_idx, assessment = assessment_idx),
    data = data
  )
}

manual_rset(
  splits = split_list,
  ids = paste0("Fold", seq_len(v))
)
}

strict_st_cv <- make_strict_spatiotemporal_splits(
  model_data,
  spatial_col = "spatial_block",
  time_col = "year",
  v = 5
)

strict_st_results <- fit_resamples(
  sst_workflow,
  resamples = strict_st_cv,
  metrics = regression_metrics,
  control = control_resamples(save_pred = TRUE)
)

collect_metrics(strict_st_results)

## # A tibble: 3 x 6
##   .metric .estimator mean      n std_err .config
##   <chr>   <chr>      <dbl> <int>   <dbl> <chr>

```

```
## 1 mae      standard    0.439      5 0.01000 pre0_mod0_post0
## 2 rmse     standard    0.596      5 0.0138  pre0_mod0_post0
## 3 rsq      standard    0.996      5 0.000270 pre0_mod0_post0

strict_st_summary <- summarize_resamples(strict_st_results, "Spatial-temporal block CV")
strict_st_summary
```

```
## # A tibble: 1 x 5
##   Evaluation          `Mean RMSE` `SE RMSE` `Mean MAE` `Mean R2`
##   <chr>              <dbl>    <dbl>    <dbl>    <dbl>
## 1 Spatial-temporal block CV      0.596    0.0138    0.439    0.996
```

Discussion: Spatial-Temporal Holdout

The spatial-temporal holdout is the hardest evaluation in this homework because the model is asked to predict observations from a region and years that were excluded from training.

If this error is much larger than the random CV error, that is not a failure of the model. It is evidence that the original random CV estimate was answering an easier question.

10. Compare All Evaluation Strategies

After you complete the temporal, spatial, and spatial-temporal sections, combine the summary tables.

```
# Combine all summaries into one table.
```

```
comparison <- bind_rows(
  random_summary,
  temporal_summary,
  spatial_summary,
  strict_st_summary
)
```

```
comparison
```

```
## # A tibble: 4 x 5
##   Evaluation          `Mean RMSE` `SE RMSE` `Mean MAE` `Mean R2`
##   <chr>              <dbl>    <dbl>    <dbl>    <dbl>
## 1 Random k-fold CV      0.463    0.00240    0.332    0.997
## 2 Blocked Temporal k-fold CV 0.596    0.0232    0.438    0.996
## 3 Blocked Spatial k-fold CV 0.472    0.0100    0.338    0.997
## 4 Spatial-temporal block CV 0.596    0.0138    0.439    0.996
```

Plot the Comparison

Complete the plotting code below. You should produce a bar chart of mean RMSE for the evaluation strategies.

```
# TODO: Create a bar chart of Mean RMSE by Evaluation strategy.
```

```
# Hint: use ggplot(comparison, aes(x = Evaluation, y = `Mean RMSE`)) + geom_col()
```

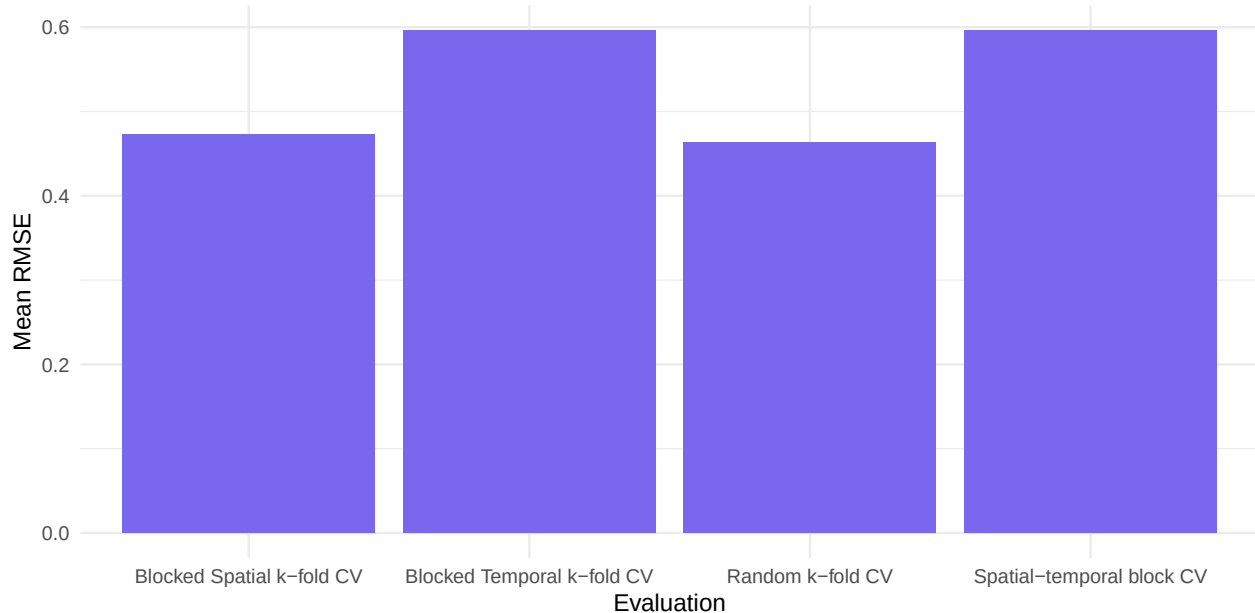
```
#ggplot(data=comparison,aes(x=Evaluation,y="Mean RMSE"))+
# geom_col()
```

```
ggplot()+
```

```
  geom_bar(data=comparison,aes(x=Evaluation,y=comparison$`Mean RMSE`),stat="identity",fill="slateblue2")
```

```
labs(x="Evaluation",y="Mean RMSE")+
theme_minimal()
```

```
## Warning: Use of `` comparison$`Mean RMSE` `` is discouraged.
## i Use `Mean RMSE` instead.
```



11. Final Reflection

Write a brief paragraph answering the following:

1. Which evaluation strategy gave the most optimistic result? #best RMSE spat-temp
2. Which evaluation strategy gave the most conservative result? #smallest mae random k fold
3. Which strategy best matches a forecasting problem?
4. Which strategy best matches prediction at unsampled locations? #r2 random k fold
5. Why should environmental scientists be cautious when reporting random cross-validation accuracy for spatial or temporal datasets?

Your answer: TODO

The problem hopes to predict the locations and time periods over large scales and the Spatial-Temporal k-fold best represents this, while also producing the most optimistic/best result based on RMSE. Random k-fold is technically the worst performing but does produce the most conservative results (smallest MAE) and can predict unsampled locations the best based on R2. This is because the broad selection of data (hemispheric, seasonal, long time series) has drowned the spatial and temporal relationship, which is one of the reasons that environmental scientists should make sure their model is accounting for the issues in their data before instead of applying random k-fold cv's for data that has a high likelihood of being spatially or temporally autocorrelated.